

Arrival and Departure: Measuring Programming Language Community Vitality with the Replacement Rate

Thiago Carvalho

2026-05-28

Abstract

Background: The long-term viability of a programming language depends on its ability to attract new practitioners faster than it loses existing ones, yet no standard metric for this exists in the software engineering literature. **Aims:** I propose *replacement rate*, the ratio of developers creating their first over their last public repository in a language in a given year, as a measure of programming language vitality, drawing on Fishman’s sociolinguistic model of language shift. **Method:** I collected public repository data from GitHub for four languages (Clojure, Elixir, OCaml, and Zig) and computed the replacement rate per language per year. **Results:** The replacement rate trends downward for all four communities over 2020–2024. Zig is the only language with a rate above 1 in 2024 ($R = 1.16$), indicating continued net growth, while Clojure shows the steepest decline ($R = 0.74$). Elixir performs comparably to Clojure despite being a younger language, while OCaml shows greater resilience than either. **Conclusions:** The replacement rate differentiates communities at distinct stages of the adoption lifecycle. The universal downward trend warrants caution: all four languages are niche communities, and whether the pattern holds for mainstream languages remains an open question.

1. Introduction

The question of whether a programming language is growing or declining is of practical importance to developers and organizations. I use *vitality* to refer to a language community’s capacity to sustain and renew its practitioner base over time — one dimension of which is whether new members are joining faster than existing ones are leaving. Existing proxies such as search engine query volume (TIOBE Index) or survey self-reports (Stack Overflow Developer Survey) measure current interest or awareness, not whether a community is replacing departing

members with new ones. I borrow an idea from sociolinguistics to propose a metric that captures renewal directly.

Joshua Fishman’s model of *language shift*, the gradual process by which a speech community moves from one language to another, identifies intergenerational transmission as the most critical factor in language vitality. When children are no longer raised speaking it at home, the community loses its most reliable renewal mechanism and begins to contract regardless of its current size.

While programming languages have no direct equivalent of parents teaching children, they face an analogous dynamic: a language that fails to attract new practitioners faster than it loses existing ones will eventually decline. I define the *replacement rate* as the ratio of developers publishing their first public repository in a language in a given year to those publishing their last. Repository creation serves as a proxy for the developer event of joining or leaving a community; since the metric focuses on each developer’s chronologically first and last repository per language, each developer contributes at most one arrival and one departure event.

This paper demonstrates that the replacement rate can be operationalized from public GitHub data in a consistent and reproducible way, and uses it to address the following research questions:

- **RQ1:** Does the replacement rate differ across programming languages with distinct community profiles?
- **RQ2:** How does the replacement rate evolve over time, and does it signal periods of community growth or decline?

2. Related Work

Prior work on programming language popularity has largely relied on aggregate signals: search engine query volume (TIOBE Index), survey self-reports (Stack Overflow Developer Survey), and repository counts. Borges et al. (2016) examined the factors driving the popularity of GitHub repositories — including stars, forks, and commit activity — finding that these signals capture attention and reuse but do not reflect whether a community is replacing departing contributors with new ones. This approach treats language popularity as a static property rather than a dynamic process, and does not examine whether a community is renewing itself. Renewal dynamics matter because a community can appear large and active while simultaneously failing to replace departing members. Replacement rate aims to detect this condition directly.

The sociolinguistic framework underlying this work originates with Fishman (1991), who developed the Graded Intergenerational Disruption Scale (GIDS) as a diagnostic tool for assessing the endangerment of natural languages. GIDS identifies Stage 6 (The last stage where children learn the language at home) as the critical threshold below which recovery becomes unlikely without active intervention. The present study operationalizes the renewal mechanism

at the core of GIDS, the ratio of new to departing members, at the level of public GitHub repositories.

3. Method

I collected public repository data from GitHub for four programming languages (Clojure, Elixir, OCaml, and Zig) using the GitHub API. These languages were selected to represent distinct positions on the community adoption lifecycle: Clojure is a mature Lisp dialect with a long-established community; Elixir is a functional language built on the Erlang VM created relatively recently; OCaml is a statically typed functional language with roots in academic research and a durable industrial presence; and Zig is an emerging systems programming language positioning itself as a modern alternative to C. The selection was also constrained by data collection feasibility (all four have GitHub repository volumes small enough to permit near-complete collection within the API limits described in Section 3.1) which limits generalizability to larger communities. For each repository owner and language, I classified repositories into four roles (First, Last, Only, and Other) to operationalize arrivals and departures. Repository creation serves as a proxy for the developer event: since the metric uses only each developer’s chronologically first and last repository per language, each developer contributes at most one arrival and one departure.

3.1 Data Collection

Repository data was collected from the GitHub Search API using a custom Node.js script built on the Octokit REST client. For each language, the script queried public repositories using the `language:X created:START..END` filter, where the time window was adjusted adaptively to stay within GitHub’s hard limit of 1,000 results per search query. Within each window, all results were retrieved via pagination (100 results per page), yielding a close approximation of the full set of public repositories for each language within the study period.

The collection covers repositories created from each language’s earliest significant GitHub presence through May 1, 2026: Clojure and OCaml from December 2008, Elixir from January 2012, and Zig from December 2015. For each repository, the following fields were retained: repository identifier, owner login and type, creation, update, and last-push timestamps, primary language as detected by GitHub, and star count.

After collection, repositories were filtered to retain only those owned by individual user accounts (excluding organizations). The final dataset comprises 300,153 repositories across 104,385 unique developers.

Two limitations follow from this approach. First, the GitHub Search API returns only public repositories, so developers who work primarily in private repositories are excluded. Second, language classification relies on GitHub’s Linguist tool, which assigns a single primary language

per repository; polyglot repositories are attributed to whichever language comprises the largest share of code.

3.2 Operationalization of the Replacement Rate

For each repository owner and language, repositories were sorted chronologically and assigned one of four roles: *First* (the owner’s earliest repository in that language), *Last* (the most recent), *Only* (the owner has exactly one repository in that language), or *Other* (all intermediate repositories, and the most recent repository if it falls within the right-censoring window). This classification was performed independently per owner–language pair, so a developer active in multiple languages receives separate First and Last labels for each.

To address right-censoring, a repository is only labeled *Last* if it was created more than one year before the collection cutoff of May 1, 2026 (i.e., before May 1, 2025). The one-year threshold is a modeling choice: a shorter window would incorrectly mark recently active developers as departed, while an indefinitely long window would fail to surface genuine departures within the study period. Repositories that are the most recent for their owner but fall within that final year are labeled *Other*, reflecting uncertainty about whether the developer has genuinely left the community.

Owners with a single repository (*Only*) are treated as both an arrival and a departure in the year of that repository’s creation, subject to the same right-censoring rule: an *Only* repository created within the last year contributes to arrivals but not departures.

The replacement rate for language l in year y is then:

$$R_{l,y} = \frac{\text{First}_{l,y} + \text{Only}_{l,y}}{\text{Last}_{l,y} + \text{Only}_{l,y}^*}$$

where $\text{Only}_{l,y}^*$ counts only *Only* repositories created before the censoring threshold. A value above 1 indicates that more developers began using the language than stopped in that year; a value below 1 indicates net departure.

The replacement rate is reported for 2020–2024. Earlier years are excluded because Zig has sparse repository counts before 2020. The full historical period for each language is examined separately in Section 4.1 to assess whether the 2020–2024 window is representative of each language’s longer-term trajectory.

4. Results

This section reports the replacement rate and related counts for the four study languages. Figure 1 shows the number of new public repositories created per year for each of the four languages since 2020. Clojure is the only language among the four with a declining trend in absolute repository creation; Zig shows the most vigorous growth, with repository counts growing 5.6x between 2020 and 2024.

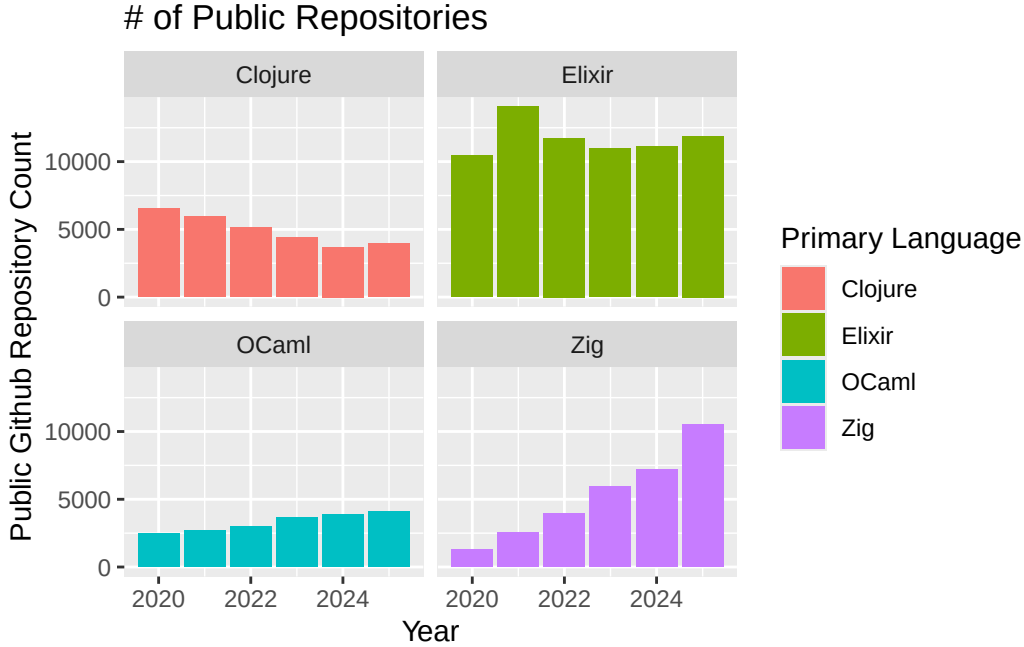


Figure 1: Public repository counts by year and language.

Figure 2 shows the replacement rate for each language from 2020 to 2024. All four communities exhibit a downward trend over the period. As of 2024, Zig is the only language with a replacement rate above 1 ($R = 1.16$), indicating that it continues to attract more new developers than it loses, though at a slower rate than in prior years. Clojure shows the steepest decline, with a 2024 replacement rate of 0.74 (Table 1), meaning that for every four developers who published their last Clojure repository, only three published their first.

Table 1: Replacement rate by language, 2024.

Language	First repos	Last repos	Replacement rate
Clojure	1268	1688	0.75
Elixir	3872	4705	0.82
OCaml	2333	2495	0.94

Table 1: Replacement rate by language, 2024.

Language	First repos	Last repos	Replacement rate
Zig	3622	3107	1.17

4.1 Historical Replacement Rate

Figure 3 extends the replacement rate to the full collection period for each language, enabling a longer-term view of community dynamics, contextualized against known events in each language’s history.

5. Discussion

The four study communities show meaningfully different replacement rates despite facing a common downward trend, suggesting the metric captures genuine variation in community health rather than a uniform effect of the period studied. All four rates declined over 2020–2024, but the spread — from Zig (above 1 throughout) to Clojure (at 0.74 by 2024) — reflects distinct trajectories that map onto each language’s known position in the adoption lifecycle. Languages where the rate falls and remains below 1 for sustained periods may be experiencing the kind of community contraction Fishman described for natural languages — a condition that population size alone would not reveal.

Addressing RQ1, the replacement rate differentiates communities with distinct profiles, though not always along the lines that age alone would predict. Zig, the youngest language in the study, shows a rate above 1, consistent with a community still in the adoption phase. OCaml and Clojure are both mature languages, yet their replacement rates diverge considerably: OCaml’s rate remains markedly higher than Clojure’s, suggesting that longevity does not straightforwardly determine community health. Most striking is Elixir: a younger language might be expected to retain the attraction of novelty, yet its replacement rate in 2024 is comparable to Clojure’s, placing it among the more troubled communities in the study rather than the healthier ones. Together, these patterns suggest that factors beyond language age, such as niche clarity, institutional adoption, and competitive pressure from adjacent languages, shape community renewal dynamics.

Addressing RQ2, the full historical view (Figure 3) reveals that the downward trend visible in the 2020–2024 window is not a recent phenomenon but a long-running pattern for the more established languages. Clojure’s replacement rate was already declining from its peak around 2011–2013, a period coinciding with Rich Hickey’s influential “Simple Made Easy” talk and the release of ClojureScript, coinciding with a surge of new arrivals that was not sustained. Elixir’s rate peaked around 2014–2015 around the time of the 1.0 release and the emergence of the Phoenix framework, then entered a sustained decline despite the language remaining

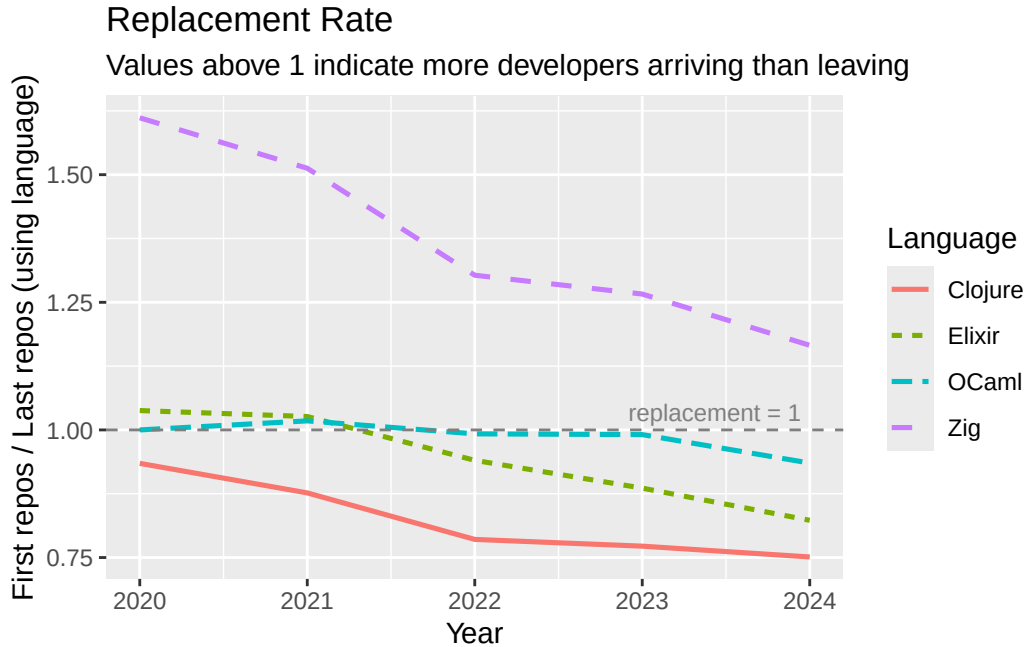


Figure 2: Replacement rate by year and language. A value above 1 indicates more developers joining than leaving.

active. OCaml’s rate shows greater stability across the period, consistent with its role as a specialist language with a durable institutional base. Zig is the only language whose full-period trajectory is unambiguously positive, with growth accelerating around the time of the Zig Software Foundation’s founding in 2020 and renewed mainstream attention in 2022.

These patterns are consistent with the replacement rate capturing real community dynamics, and they complicate the concern that the 2020–2024 window is unrepresentative — the downward trend for Clojure and Elixir predates that window and is therefore not an artefact of studying a particular five-year period. It should be noted, however, that the events annotated in Figure 3 were identified by examining the charts: a more rigorous validation would require predicting inflection points from independently documented events before examining the rate data. Whether the same pattern holds for larger mainstream communities — such as Rust, Python, or TypeScript — remains an open question and a priority for future work.

The Fishman analogy illuminates the mechanism — renewal is more diagnostic of long-term health than population size — but breaks down in several respects. Most importantly, natural language shift is typically costly to reverse once intergenerational transmission fails, and the community may not survive as a distinct entity. Programming language communities face different constraints: a language can revive when it finds a new application domain, receives institutional backing, or is adopted by a new generation of developers for reasons unrelated to its prior community. Developers also routinely use multiple languages simultaneously, and

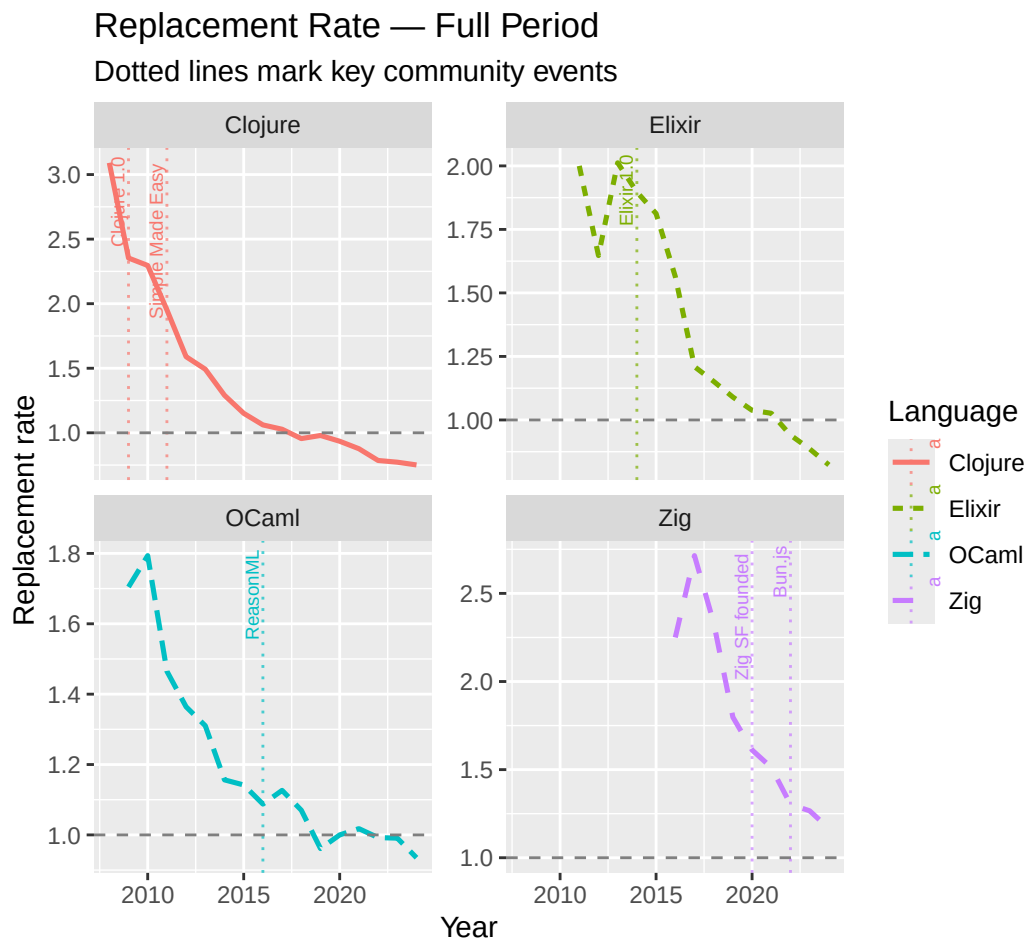


Figure 3: Replacement rate over the full collection period. Dotted vertical lines mark key events in each language's history.

switching costs are far lower than for natural languages. The replacement rate measures the current trajectory, not the ceiling.

6. Threats to Validity

Construct validity: The replacement rate uses a developer’s first and last public repository as proxies for joining and leaving a community. A developer may continue using a language privately after their last public commit, inflating apparent departures. Conversely, a developer may create public tutorial or experimental repositories without ever becoming a sustained community member, inflating apparent arrivals.

Internal validity: The GitHub API returns only public repositories. Differential rates of private repository use across languages could bias comparisons. This concern applies particularly to Clojure (with significant adoption at Nubank) and OCaml (with significant adoption at Jane Street), whose replacement rates may be more severely underestimated relative to Zig and Elixir, which have stronger open-source orientations.

External validity: Results are based on four languages selected for distinct community profiles and data collection feasibility — all four are niche communities with GitHub repository volumes small enough to permit near-complete collection. This introduces selection bias: niche communities may behave differently from mainstream languages such as Python or TypeScript, and the uniformly declining trend observed here may reflect characteristics specific to this size class of community rather than a universal pattern.

Conclusion validity: The replacement rate is a descriptive ratio and no inferential statistical tests are applied in this study. Observed differences between languages and across years should be interpreted as patterns in the data rather than statistically significant effects. Future work should establish confidence intervals, accounting for the year-to-year autocorrelation in developer activity: a developer who was a *First* in year y remains in the community through year $y + 1$, $y + 2$, and so on, so the numerator and denominator of the replacement rate in adjacent years are not independent.

7. Conclusion

Applied to GitHub data for Clojure, Elixir, OCaml, and Zig, the replacement rate surfaces meaningful differences in community trajectory that aggregate popularity measures do not capture: Zig is growing while Clojure contracts, and OCaml’s institutional niche affords it more stability than Elixir despite being older. I proposed the replacement rate — the ratio of developers creating their first versus their last public repository in a language — as one measurable dimension of programming language vitality, drawing on Fishman’s insight that renewal is more diagnostic of long-term health than population size. Future work should extend the analysis to mainstream languages such as Rust, Python, and TypeScript to determine whether the downward trends observed here are specific to niche communities. Establishing

the metric’s validity against independent ground truth — such as survey-based community assessments or longitudinal developer panel data — would strengthen confidence in its interpretability. Inferential methods, including confidence intervals that account for year-to-year autocorrelation in developer activity, would strengthen the statistical grounding of the metric. The approach could also draw on alternative data sources to capture private activity, be weighted by developer activity level, or be enriched with dependency graph data to capture indirect community influence.

Data Availability

Dataset and analysis available at <https://github.com/thiagooak/programming-language-vitality>

References

- Borges, H., Hora, A., & Valente, M. T. (2016). Understanding the factors that impact the popularity of GitHub repositories. In *Proceedings of the 32nd IEEE International Conference on Software Maintenance and Evolution* (pp. 334–344). IEEE.
- Fishman, J. A. (1991). *Reversing Language Shift: Theoretical and Empirical Foundations of Assistance to Threatened Languages*. Multilingual Matters, Clevedon, UK.

Appendix

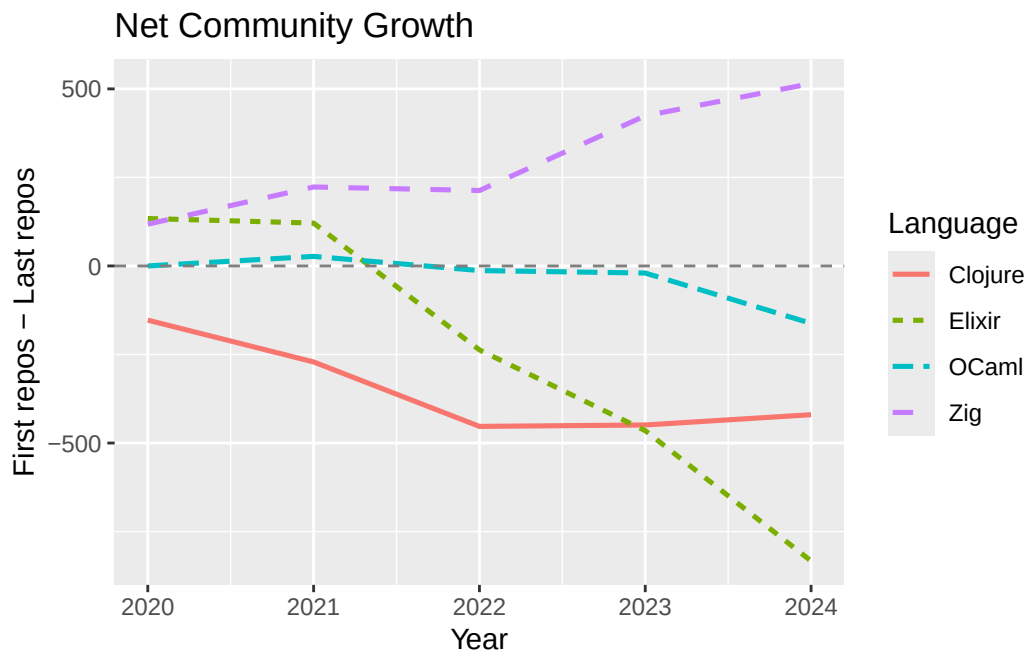


Figure 4: Net community growth (first repos minus last repos) per year. Included for reference; the replacement rate in Figure 2 normalizes this by community size.